Statistical Machine Learning Final Project: Predictions of Flight Delays at the Pittsburgh Airport

Ellie Huang (elliehua) Nikita Jayaprakash (njayapra)

Executive summary

1 Introduction

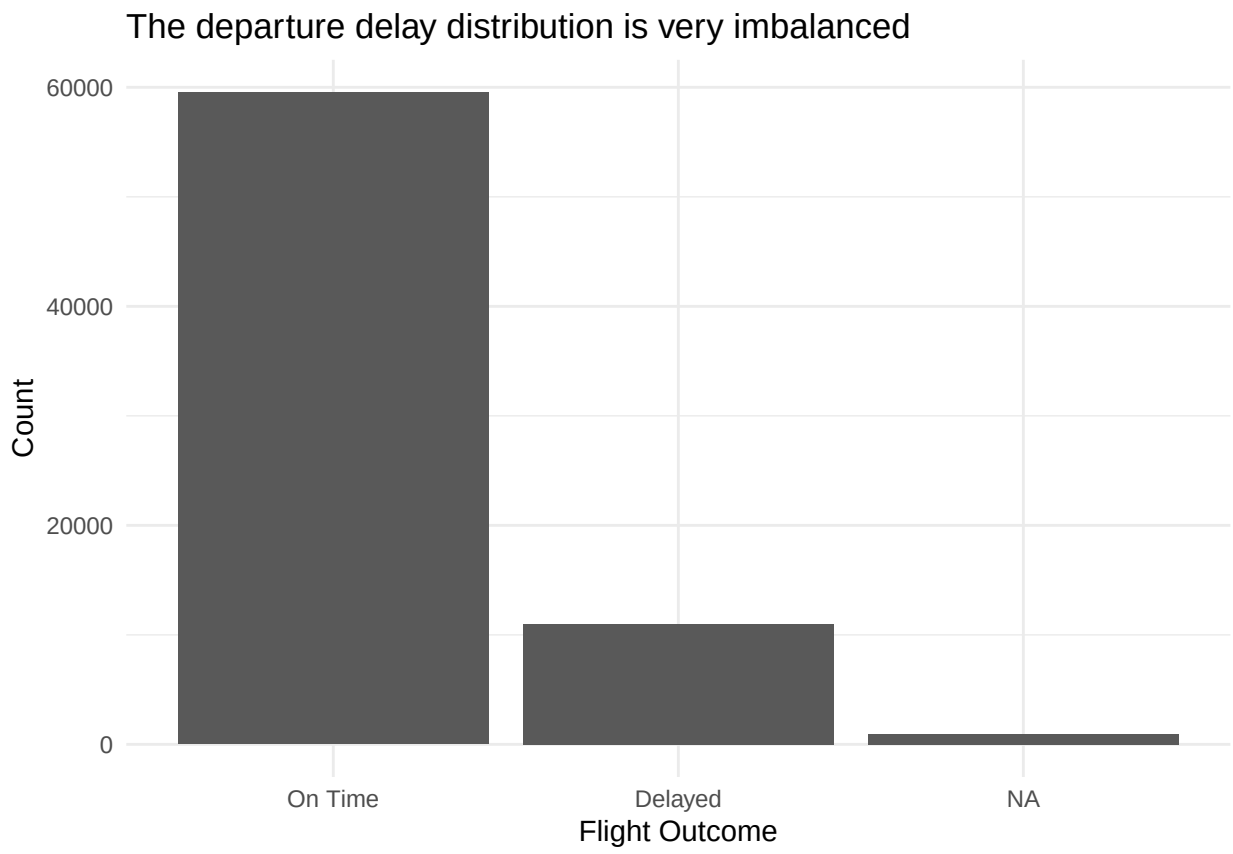
The goal of this analysis is to predict whether or not a departing flight from Pittsburgh International Airport (airport code PIT) will be delayed based on only information available before the departure of the flight. The dataset is flight information from 2023 - 2025 extracted from the Airline On-Time Performance Data from the Bureau of Transportation Statistics of the U.S. Department of Transportation. In addition to metrics about flight delays, it also reflects commercial flight activity to and from PIT. Feature engineering was used to develop additional metrics based on initial exploratory data analysis. This analysis utilizes various machine learning methods in order to attempt this binary classification on a subset of the 2025 flights, including

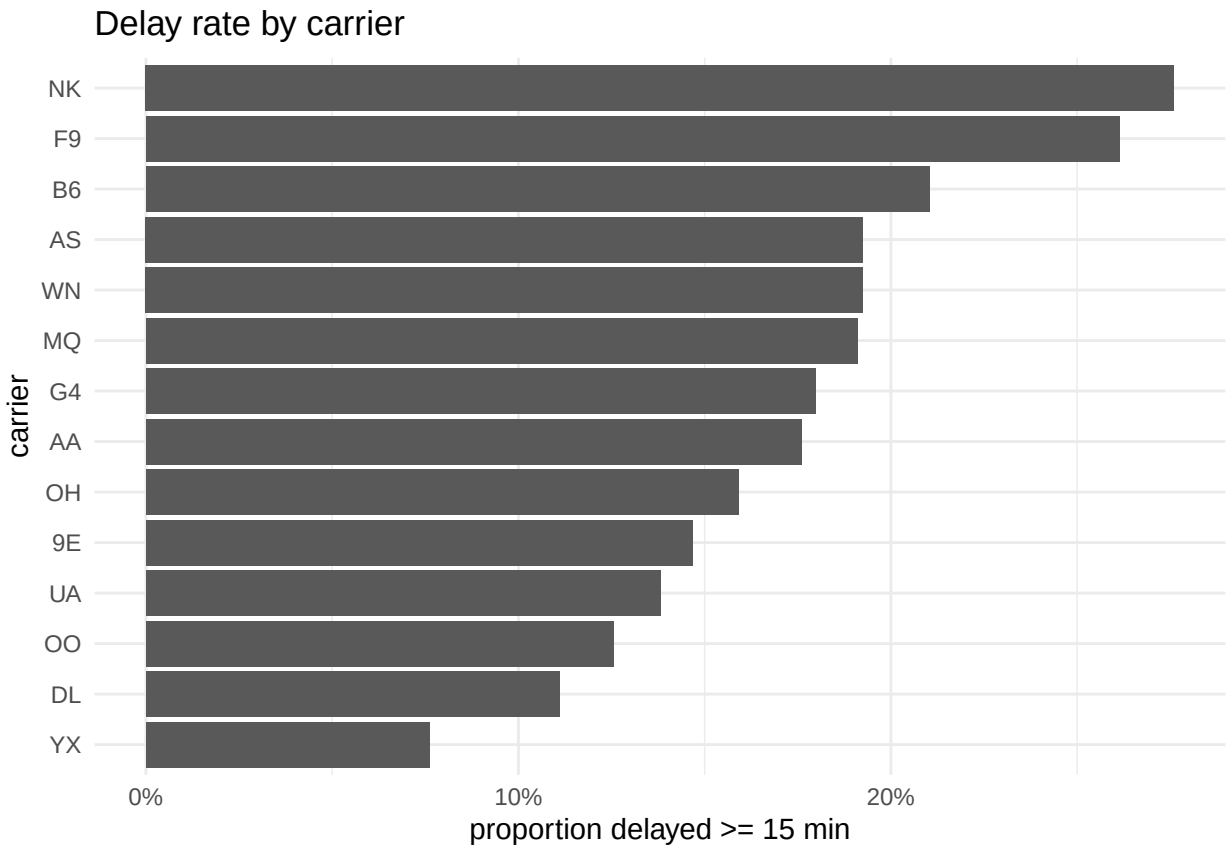
- logistic regression with and without regularizatoin
- XGBoost
- random forest with and without rebalancing
- an average of the predictions of these metrics with and without weighting different methods

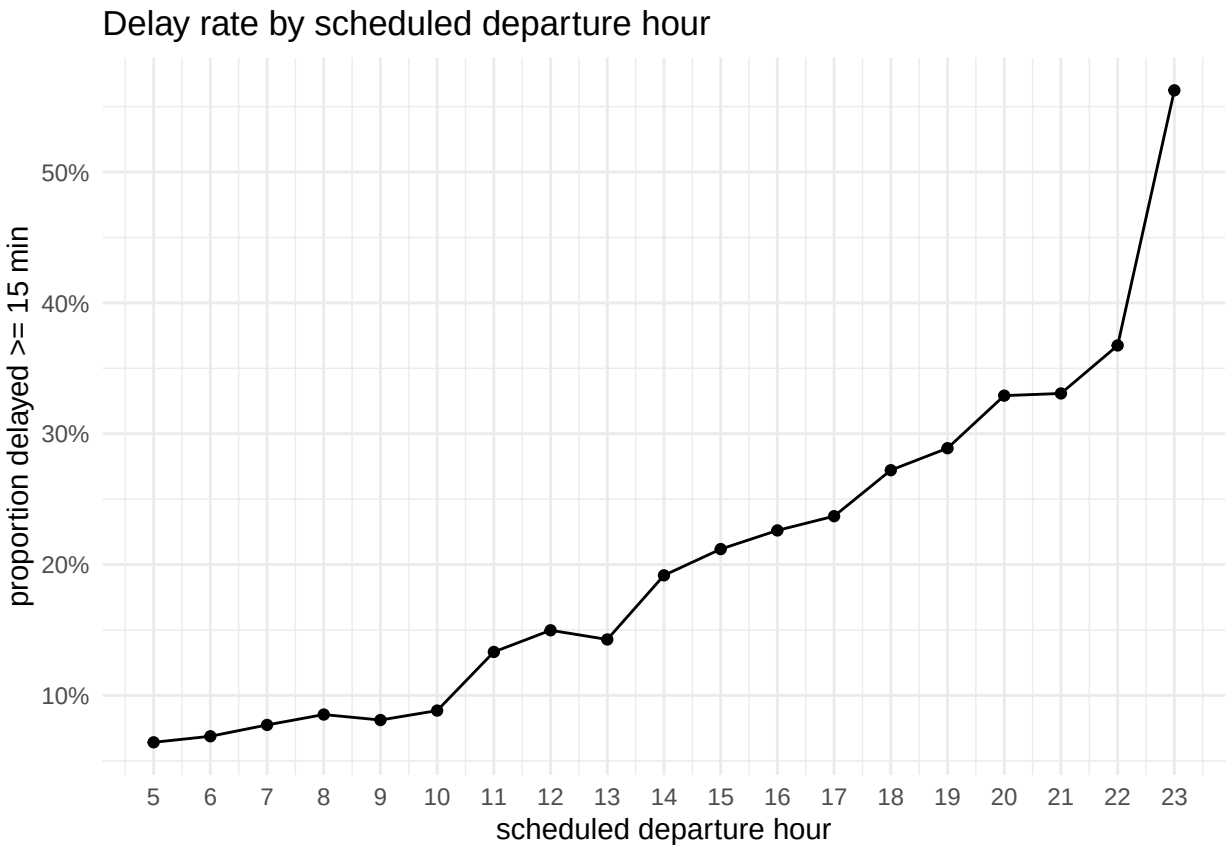
The metrics of model performance are training set misclassification rate, testing set misclassification rate, and area-under-the-curve (AUC). Based on the findings, ... [insert conclusions here].

2 Exploration

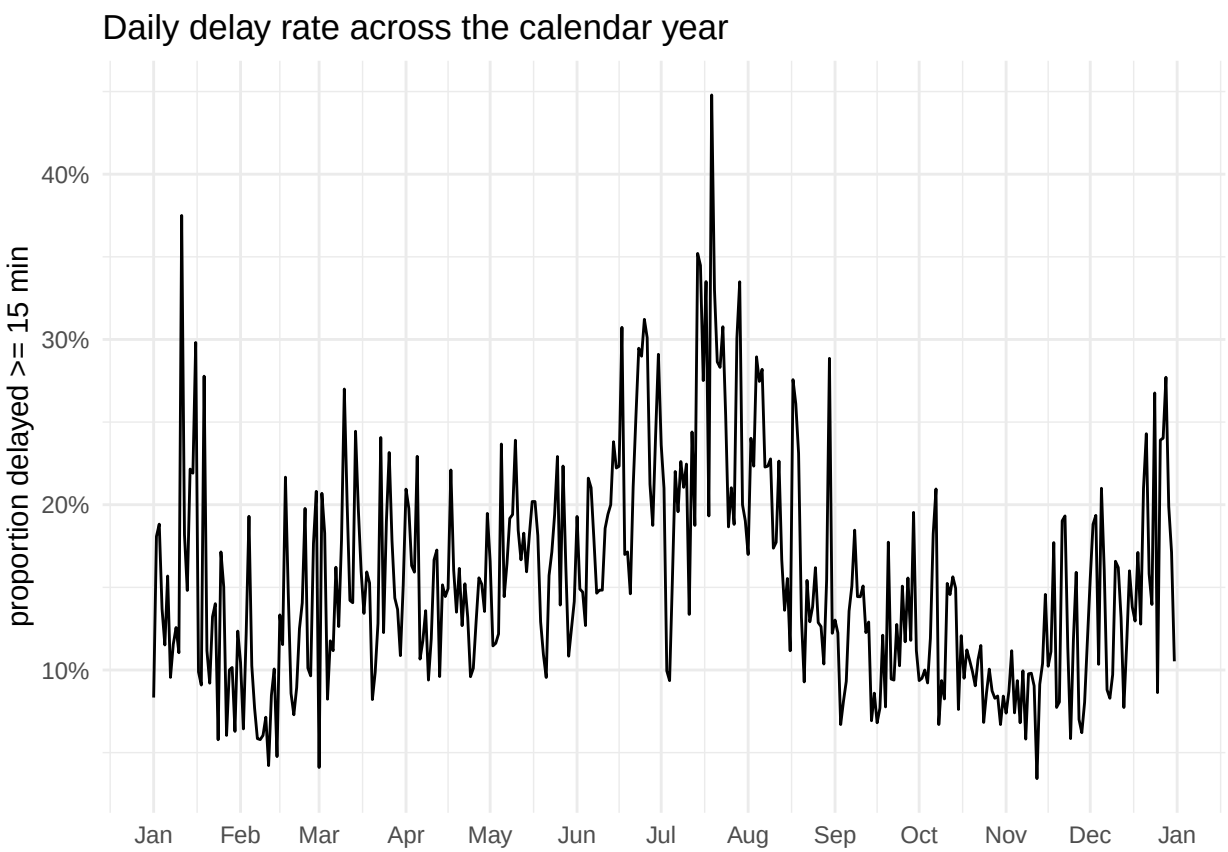
2.1 Exploratory Data Analysis

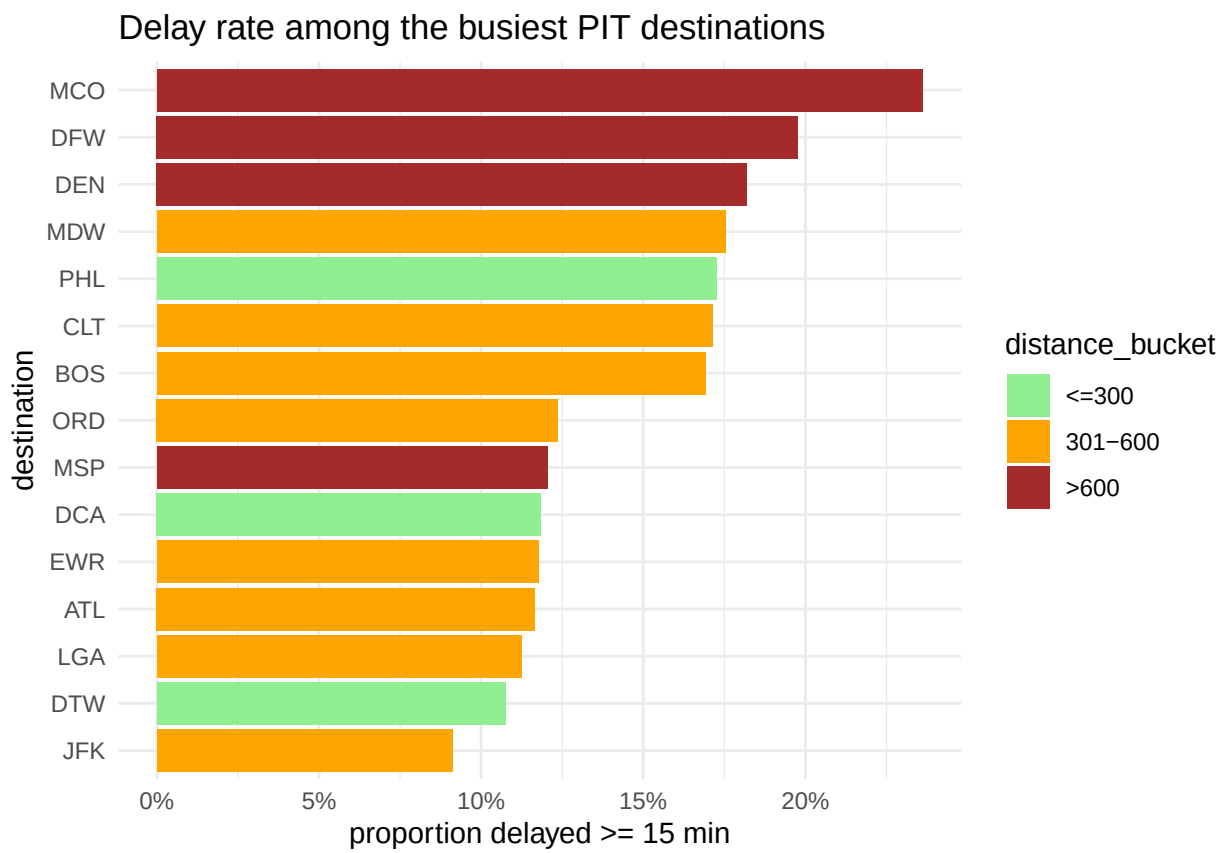



Delay rate by carrier
NK
F9
B6
AS
WN
MQ
G4
AA
OH
9E
UA
OO
DL
YX
carrier
0%
10%
20%
proportion delayed >= 15 min


Delay rate by scheduled departure hour
proportion delayed >= 15 min
50%
40%
30%
20%
10%
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
scheduled departure hour

Flight Date	Delay Rate	Count
November 20	0.3750000	176
April 20	0.2894737	190
June 20	0.2819149	188
May 20	0.2746114	193
October 20	0.2700000	200
December 20	0.2440191	209
February 20	0.2401961	204
October 20	0.2390244	205
May 20	0.2367150	207
January 20	0.2361809	199





2.2 Feature Engineering

3 Supervised Analysis

4 Analysis of Results

Appendix

```r
###########################
# STYLE EDITS: IGNORE THIS
###########################

# normally you'll want to include this with the libraries at the beginning of your document
knitr::opts_chunk$set(message = FALSE) # include this if you don't want markdown to knit messages
knitr::opts_chunk$set(warning = FALSE) # include this if you don't want markdown to knit warnings
knitr::opts_chunk$set(echo = FALSE) # set echo = FALSE to hide code from output

library(tidyverse)
library(adabag)
library(caret)
library(e1071)
library(tidymodels)
library(here)
library(janitor)
library(lubridate)
library(pROC)
library(randomForest)
library(skimr)
library(scales)
library(forcats)
library(ranger)
library(glmnet)
library(xgboost)
# Load data
flights2023 <- read_csv(here("data", "flights2023.csv")) |>
 clean_names()

flights2024 <- read_csv(here("data", "flights2024.csv")) |>
 clean_names()

test <- read_csv(here("data", "flights2025.csv")) |>
 clean_names()

test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

# Build training set
train <- bind_rows(
 flights2023 |> mutate(data_year = 2023),
 flights2024 |> mutate(data_year = 2024)
)
# Apply the same cleaning steps to each data frame so train and test sets are consistent
clean_flights <- function(df) {
 df |>
 clean_names() |>
```

```r
 mutate(
 fl_date = as.Date(fl_date),
 dep_del15 = as.numeric(dep_del15),
 op_unique_carrier = as.factor(op_unique_carrier),
 origin = as.factor(origin),
 dest = as.factor(dest),
 sched_dep_hour = floor(crs_dep_time / 100),
 sched_arr_hour = floor(crs_arr_time / 100),
 month = as.factor(month),
 day_of_week = as.factor(day_of_week)
 )
}

train <- clean_flights(train)
test <- clean_flights(test)
test_visible <- clean_flights(test_visible)
test_guess <- clean_flights(test_guess)
# Restrict to departures from PIT
train <- train |>
 filter(origin == "PIT")

test_visible <- test_visible |>
 filter(origin == "PIT")

test_guess <- test_guess |>
 filter(origin == "PIT")

# Quick check of the target distribution
table(train$dep_del15, useNA = "ifany")
# Inbound flight features
# For each PIT departure, we try to match the same aircraft (tail + date) arriving at PIT
# earlier that day. Match rates: ~55% for training, ~15% for guess flights.
# For guess, only arrivals in flights2025_visible.csv have known arr_delay -
# guess-period arrivals are blanked out. The 85% with no match are overnighted
# or first-flight aircraft, which is itself informative (lower delay risk).
arrivals_train <- bind_rows(
  clean_flights(read_csv(here("data", "flights2023.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2023),
  clean_flights(read_csv(here("data", "flights2024.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2024)
)

# Only visible arrivals have non-NA arr_delay bc guess-period arrivals are blanked out
arrivals_test_visible <- clean_flights(
  read_csv(here("data", "flights2025_visible.csv"), show_col_types = FALSE)
) |> filter(dest == "PIT") |> mutate(data_year = 2025)

# For each (date, tail_num), keep the latest arriving flight as the inbound leg
make_inbound_lookup <- function(arrivals_df) {
  arrivals_df |>
    filter(!is.na(tail_num), tail_num != "", !is.na(arr_delay)) |>
    group_by(fl_date, tail_num) |>
    slice_max(crs_arr_time, n = 1, with_ties = FALSE) |>
```

```r
    ungroup() |>
    select(fl_date, tail_num,
           inbound_arr_delay = arr_delay,  # minutes late on arrival
           inbound_arr_del15 = arr_del15,  # binary: arrived >= 15 min late
           inbound_crs_arr_time = crs_arr_time)  # scheduled arrival time
}

inbound_train <- make_inbound_lookup(arrivals_train)
inbound_visible <- make_inbound_lookup(arrivals_test_visible)
inbound_guess <- inbound_visible  # guess set uses visible-only lookup

join_inbound <- function(dep_df, inbound_df) {
  dep_df |>
    left_join(inbound_df, by = c("fl_date", "tail_num")) |>
    mutate(
      # min between scheduled inbound arrival and scheduled departure
      turnaround_mins = sched_dep_hour * 60 + (crs_dep_time %% 100) -
                          (inbound_crs_arr_time %/% 100 * 60 + inbound_crs_arr_time %% 100),
      # indicator (< 45 min scheduled buffer is risky)
      tight_turnaround = if_else(!is.na(turnaround_mins) & turnaround_mins < 45, 1L, 0L),
      # indicator (whether we found a matching inbound leg at all)
      # has_inbound = 0 could mean overnighted OR inbound after cutoff in guess set
      # so we keep it as context but don't include it in model_vars
      has_inbound = if_else(!is.na(inbound_arr_delay), 1L, 0L),
      # fill unmatched rows with 0 (no evidence of inbound delay)
      inbound_arr_delay = coalesce(as.numeric(inbound_arr_delay), 0),
      inbound_arr_del15 = coalesce(as.numeric(inbound_arr_del15), 0),
      turnaround_mins = coalesce(turnaround_mins, 0)
    )
}

train <- join_inbound(train, inbound_train)
test_visible <- join_inbound(test_visible, inbound_visible)
test_guess <- join_inbound(test_guess, inbound_guess)
train |>
  ggplot(aes(x = factor(dep_del15,
                        labels = c("On Time", "Delayed")))) +
  geom_bar() +
  labs(
    title = "The departure delay distribution is very imbalanced",
    x = "Flight Outcome",
    y = "Count"
  ) +
  theme_minimal()
carrier_delay_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(op_unique_carrier) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(desc(delay_rate))
```

```r
carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by carrier",
 x = "carrier",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
hour_delay_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 )

hour_delay_tbl |>
 ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
 geom_line() +
 geom_point() +
 scale_x_continuous(breaks = 0:23) +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by scheduled departure hour",
 x = "scheduled departure hour",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
# Delay rate by calendar date, aligned to a common year for display
train |>
 filter(!is.na(dep_del15)) |>
 mutate(plot_date = make_date(year = 2010, month = as.integer(as.character(month)), day = day_of_month)
 group_by(plot_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = plot_date, y = delay_rate)) +
 geom_line() +
 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Daily delay rate across the calendar year",
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
bad_days_tbl <- train |>
 filter(!is.na(dep_del15)) |>
```

```r
 group_by(fl_date) |>
 summarise(
 `Delay Rate` = mean(dep_del15),
 Count = n(),
 .groups = "drop"
 ) |>
 filter(Count >= 10) |>
 arrange(desc(`Delay Rate`))
bad_days_tbl |>
  mutate(`Flight Date` = format(as.Date(fl_date,
                                        format = "%M-%d-%y"), "%B %d")
         ) |>
  head(10) |>
  select(`Flight Date`, `Delay Rate`, Count) |>
  gt::gt()
top_dests <- train |>
 count(dest, sort = TRUE) |>
 slice_head(n = 15) |>
 pull(dest)

train <- train |>
 mutate(
  distance_bucket = case_when(
  distance > 600 ~ ">600",
  distance > 300 ~ "301-600",
  TRUE ~ "<=300"
 ),
 distance_bucket = factor(
 distance_bucket,
 levels = c("<=300", "301-600", ">600")
 )
 )

train |>
 filter(!is.na(dep_del15), dest %in% top_dests) |>
 group_by(dest, distance_bucket) |>
 summarise(
   delay_rate = mean(dep_del15),
   n = n(),
   .groups = "drop"
 ) |>
  arrange(delay_rate) |>
  ggplot(aes(x = reorder(dest, delay_rate), y = delay_rate,
             fill = distance_bucket)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "Delay rate among the busiest PIT destinations",
    x = "destination",
    y = "proportion delayed >= 15 min"
    ) +
  theme_minimal() +
```

```r
scale_fill_manual(values = c("lightgreen", "orange", "brown"))
```